

Relatório de Extensão modalidade Prestação de Serviço – Projeto de Software

Nome / Título do Sistema: Juliu's Hot Dog - 2º fase (Desenvolvimento Back-end)

Equipe: Eduardo Lemes / Ruan Almagro

Público-alvo: Clientes do food truck Julius Hot Dog, principalmente estudantes do IFSP e CETEC, além do próprio Julius como administrador do sistema.

Carga-horária total da ação: 40 horas

Importância extensionista: O projeto foi desenvolvido a partir da observação direta do funcionamento do food truck Julius Hot Dog, identificando dificuldades reais como demora no atendimento, falta de controle de estoque e ausência de um cardápio digital.

A solução proposta tem grande impacto social e prático, pois melhora a experiência dos clientes, reduz erros operacionais e auxilia o empreendedor a organizar seu negócio, especialmente em horários de pico. Além disso, fortalece a relação entre tecnologia e comunidade local.

Plataforma do sistema (desktop, web, mobile, ou mais de uma): Sistema Web

Objetivo Geral: Desenvolver um sistema web para otimizar o processo de pedidos, controle de estoque e atendimento ao cliente do Julius Hot Dog.

Objetivos Específicos (quando for o caso):

- Criar um cardápio online interativo
- Permitir personalização de pedidos
- Implementar controle de estoque por insumos
- Organizar pedidos por horário de retirada
- Integrar envio de pedidos via WhatsApp
- Disponibilizar painel administrativo para gestão

Metodologia e Avaliação do ciclo de vida (software):

A metodologia adotada para o desenvolvimento do sistema será baseada em uma abordagem incremental, inspirada em práticas ágeis, permitindo a construção do software de forma progressiva e organizada. Inicialmente, foi realizado o levantamento de requisitos, seguido da modelagem do sistema, onde foram definidos os principais componentes e fluxos. Em seguida, será efetuado o desenvolvimento por partes, implementando funcionalidades gradualmente. Durante todo o processo, serão realizados testes contínuos para garantir o correto funcionamento das funcionalidades, finalizando com ajustes e melhorias conforme necessário.

A avaliação do sistema será conduzida com base em critérios práticos e funcionais, visando garantir sua qualidade e eficiência. Serão realizados testes de funcionamento das funcionalidades implementadas, além de simulações de pedidos para validar o fluxo completo do sistema. Também será feita a verificação da usabilidade, analisando se a interface é intuitiva e de fácil utilização. Por fim, o sistema será validado junto ao cliente, assegurando que a solução atenda às necessidades reais do negócio.

Especificações técnicas:

1) Ciclo de Vida do Desenvolvimento Back-end

O desenvolvimento do back-end do sistema Julius Hot Dogs segue um ciclo de vida organizado em etapas, permitindo que a construção da aplicação seja realizada de forma estruturada, iniciando pela definição da arquitetura, passando pela modelagem e implementação do banco de dados, desenvolvimento das APIs, criação das regras de negócio e finalizando com testes e validações.

Cada etapa possui uma carga horária definida de acordo com sua complexidade e importância dentro do desenvolvimento da solução.

Etapa	Atividade	Descrição Detalhada	Horas
1	Planejamento Técnico do Back-end	Definição da arquitetura da aplicação, organização do projeto, escolha das tecnologias utilizadas (Node.js, Express.js, JavaScript, Supabase/PostgreSQL e WhatsApp Cloud API da Meta), definição dos padrões de comunicação entre front-end, API e banco de dados.	4h
2	Modelagem do Banco de Dados	Criação da estrutura de dados do sistema, definição das entidades principais (usuários, produtos, pedidos, itens do pedido, insumos e estoque), relacionamentos e regras de integridade.	5h
3	Implementação e Integração do Banco de Dados	Criação das tabelas no banco PostgreSQL, configuração da conexão com o backend, criação das operações de inserção, consulta, atualização e exclusão dos dados.	6h
4	Desenvolvimento da API RESTful	Desenvolvimento dos endpoints responsáveis pela comunicação do sistema, incluindo cadastro de usuários, autenticação, gerenciamento de produtos, pedidos e controle de estoque.	7h
5	Implementação das Regras de Negócio	Desenvolvimento da lógica interna do sistema, incluindo validação dos pedidos, autorização pelo administrador, atualização automática do estoque e controle de disponibilidade dos produtos.	5h
6	Desenvolvimento do Painel Administrativo	Implementação das funcionalidades utilizadas pelo Julius, como gerenciamento de produtos, acompanhamento de pedidos, controle de estoque e visualização das informações operacionais.	6h

7	Segurança e Controle de Acesso	Implementação das validações de usuários, permissões administrativas, proteção das rotas da API e tratamento seguro das informações armazenadas.	3h
8	Testes e Validação do Back-end	Realização de testes das rotas, integração com banco de dados, verificação das respostas da API, identificação e correção de erros encontrados durante o desenvolvimento.	4h

Carga horária total prevista: 40 horas

2) Estrutura Back-end do Sistema Julius Hot Dogs

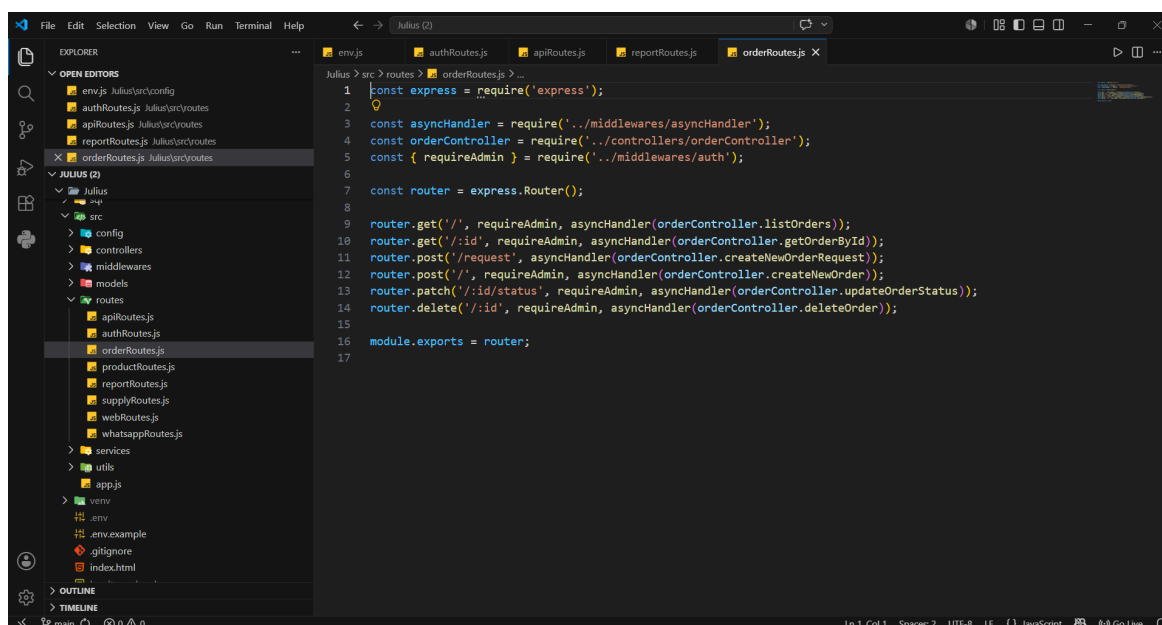
Durante o desenvolvimento do sistema Julius Hot Dogs, uma das principais preocupações foi estruturar uma base sólida para que todas as funcionalidades planejadas pudessem ser implementadas de maneira organizada, segura e escalável. Para isso, foi definida uma arquitetura de back-end responsável por intermediar a comunicação entre o usuário, as regras de negócio e o banco de dados.

O back-end funciona como o núcleo operacional do sistema, sendo responsável por receber solicitações do front-end, validar informações, executar processos internos e retornar respostas adequadas. Dentro do contexto do Julius Hot Dogs, essa camada é essencial para controlar todo o fluxo da operação, desde o cadastro de clientes até o gerenciamento de pedidos, produtos e estoque.

Para garantir uma melhor organização do desenvolvimento, o projeto foi estruturado utilizando uma arquitetura baseada em rotas, controladores, modelos e serviços. Essa divisão permite que cada parte do sistema tenha uma responsabilidade específica, facilitando a manutenção, testes e futuras expansões do sistema.

A API desenvolvida utiliza o conceito RESTful, onde cada recurso do sistema possui seus próprios endpoints responsáveis pelas operações de criação, consulta, atualização e exclusão de dados. Dessa forma, o sistema consegue representar digitalmente os processos realizados atualmente no food truck, mantendo a simplicidade operacional e permitindo que novas funcionalidades sejam adicionadas futuramente.

1. Estrutura das Rotas da API



2. Organização e Funcionamento das Rotas

A estrutura do back-end foi organizada separando as funcionalidades do sistema em diferentes módulos de rotas. Cada arquivo dentro da pasta routes representa um conjunto de operações relacionadas a uma determinada área do sistema.

Essa divisão permite que o código permaneça organizado e facilita o crescimento da aplicação, evitando que todas as funcionalidades fiquem concentradas em um único arquivo.

O fluxo de funcionamento segue o modelo:

Requisição do usuário → Rota → Controller → Model → Banco de Dados → Resposta da API

Ou seja, quando um usuário realiza uma ação no sistema, como realizar login ou criar um pedido, a solicitação é encaminhada para a rota correspondente. A rota direciona a ação para o controlador responsável, que executa as regras necessárias e realiza a comunicação com o banco de dados.

3. authRoutes.js — Rotas de Autenticação

O módulo de autenticação é responsável pelo gerenciamento dos usuários do sistema, controlando o acesso e identificando os diferentes perfis existentes.

Dentro do Julius Hot Dogs, existem dois principais usuários:

- Cliente;
- Administrador (Julius).

As rotas desse módulo são utilizadas principalmente nas telas de cadastro e login.

3. 1 Cadastro de usuário

Responsável por criar novos clientes no sistema.

Exemplo:

POST /auth/register

Utilizado na tela: **Cadastro de Cliente**

Dados enviados:

- nome;
- telefone;
- email;
- senha.

3. 2 Login

Responsável por validar os dados de acesso.

Exemplo:

POST /auth/login

Utilizado na tela: **Login do Cliente / Administrador**

Após a autenticação, o sistema identifica o tipo de usuário e libera as permissões correspondentes.

4. productRoutes.js — Gerenciamento de Produtos

Essa rota controla todas as operações relacionadas ao cardápio do Julius Hot Dogs.

Ela é utilizada tanto pelo cliente quanto pelo administrador.

Para o cliente, permite consultar os produtos disponíveis.

Para o administrador, permite gerenciar o cardápio.

4. 1 Cliente

Tela: **Cardápio**

Consulta produtos:

GET /products

Retorna informações como:

- nome;
- descrição;
- preço;
- imagem;
- disponibilidade.

Exemplo:

```
{  
  "nome": "Hot Dog Especial",  
  "preco": 18.00,  
  "disponivel": true  
}
```

4. 2 Administrador

Tela: **Gerenciamento de Produtos**

Operações disponíveis:

Criar produto:

POST /products

Atualizar:

PUT /products/:id

Excluir:

DELETE /products/:id

Essa área permite que Julius mantenha o cardápio atualizado.

5. orderRoutes.js — Gerenciamento de Pedidos

A rota de pedidos representa uma das principais funcionalidades do sistema, pois controla todo o fluxo da venda.

Ela conecta diretamente:

Cliente → Pedido → Administrador

5. 1 Criação de Pedido

Quando o cliente finaliza uma compra:

POST /orders

O sistema registra:

- cliente;
- produtos escolhidos;
- quantidade;
- valor total;
- status do pedido.

Inicialmente o pedido fica como:

PENDENTE

5. 2 Consulta de Pedido

Permite visualizar informações de uma venda específica.

GET /orders/:id

Utilizado para acompanhar:

- detalhes do pedido;
- horário;
- situação atual.

5. 3 Alteração de Status

Responsável pela decisão administrativa.

PATCH /orders/:id/status

O administrador pode alterar:

AUTORIZADO

NEGADO

Exemplo:

Julius recebe o pedido:

Pedido #15

2 Hot Dogs

Após verificar disponibilidade:

Autoriza:

AUTORIZADO

ou recusa:

NEGADO

6. supplyRoutes.js — Controle de Estoque

Essa rota representa o gerenciamento dos insumos utilizados na produção.

Ela é responsável por controlar ingredientes como:

- pão;
- salsicha;
- molhos;
- adicionais.

Tela: **Controle de Estoque**

Principais operações:

- Adicionar estoque:
 - POST /supply
- Atualizar quantidade:
 - PATCH /supply/:id
- Consultar estoque:
 - GET /supply

Durante uma venda autorizada, o sistema poderá realizar a baixa automática dos insumos utilizados.

7. reportRoutes.js — Relatórios

Responsável pela geração de informações administrativas.

Tela: **Dashboard Administrativo**

Possibilita visualizar:

- quantidade de pedidos;
- produtos mais vendidos;
- faturamento;
- movimentação do estoque.

Exemplos:

GET /reports/sales

GET /reports/products

Essas informações auxiliam Julius na tomada de decisões.

8. whatsappRoutes.js — Integração com WhatsApp

Essa rota tem como objetivo integrar o sistema com o fluxo atual de comunicação utilizado pelo Julius.

Após o cliente realizar um pedido, o sistema pode gerar uma mensagem contendo:

- cliente;
- pedido;
- produtos;
- horário de retirada.

Exemplo:

POST /whatsapp/send

A funcionalidade busca manter uma característica importante do negócio: o contato direto entre cliente e comerciante.

9. apiRoutes.js — Controle Geral da API

Essa rota funciona como uma camada geral do sistema.

Ela pode ser utilizada para:

- verificar funcionamento do servidor;
- disponibilizar informações gerais;
- controlar versões da API.

Exemplo:

GET /api/status

Retorno:

```
{  
  
  "status": "online"  
  
}
```

10. webRoutes.js — Serviços Web

Responsável por rotas relacionadas às páginas públicas ou integrações externas.

Pode atender funcionalidades como:

- informações do estabelecimento;
- horários;
- localização;
- conteúdos institucionais.

3) Considerações Finais

A organização das rotas do back-end permite que o Julius Hot Dogs seja transformado em um sistema modular, onde cada funcionalidade possui sua própria responsabilidade. Essa separação facilita o desenvolvimento, manutenção e evolução da aplicação.

Com essa arquitetura, o sistema consegue atender o funcionamento atual do food truck, digitalizando processos como pedidos, controle de estoque e gerenciamento administrativo, sem remover a característica principal do negócio: a proximidade entre Julius e seus clientes.

A estrutura criada também permite futuras expansões, como pagamentos digitais, aplicativo mobile, integração completa com WhatsApp Business e novos pontos de venda.

Resultados Obtidos:

O desenvolvimento do backend do sistema Julius Hot Dogs resultou na criação de uma base estruturada para suportar as principais operações do sistema, com foco na organização de dados, comunicação entre aplicação e banco de dados e preparação das regras de negócio.

Durante o desenvolvimento, foi implementada uma arquitetura organizada em camadas, separando responsabilidades entre rotas, controladores, modelos e serviços, permitindo maior facilidade de manutenção e evolução do sistema. A criação da API possibilitou estruturar os principais fluxos da aplicação, como cadastro de usuários, gerenciamento de produtos, controle de pedidos e integração com o estoque.

A modelagem do banco de dados permitiu armazenar informações essenciais para o funcionamento do sistema, relacionando clientes, produtos, pedidos e insumos. Essa estrutura possibilita futuras implementações como baixa automática de estoque, geração de relatórios e acompanhamento histórico das vendas.

Outro resultado importante foi a preparação das rotas responsáveis pelo gerenciamento das funcionalidades administrativas, permitindo que o sistema tenha controle sobre autorizações de pedidos, atualização de status e manutenção dos dados cadastrados.

Dessa forma, os resultados obtidos no backend estão alinhados aos objetivos iniciais do projeto, criando uma estrutura funcional, escalável e preparada para integração com o frontend, possibilitando a evolução do sistema para uma aplicação completa.

Lições Aprendidas:

O desenvolvimento do backend do sistema Julius Hot Dogs proporcionou aprendizados importantes não apenas relacionados à programação, mas também à análise e compreensão de um sistema aplicado a uma situação real. Durante o projeto, foi possível perceber que a construção de uma solução eficiente depende de uma boa compreensão do funcionamento do negócio antes mesmo da implementação do código.

Uma das principais lições aprendidas foi a importância da organização da arquitetura do sistema. A separação entre rotas, controladores, modelos e serviços mostrou-se essencial para manter o código mais limpo, facilitar a correção de erros e permitir que novas funcionalidades sejam adicionadas futuramente sem comprometer a estrutura existente.

Outro aprendizado relevante foi relacionado ao desenvolvimento de APIs e à comunicação entre diferentes partes do sistema. A criação das rotas demonstrou como o backend atua como uma ponte entre o usuário e o banco de dados, recebendo informações, validando regras e retornando respostas para o frontend. Esse processo reforçou a importância de criar endpoints bem definidos e com responsabilidades claras.

Além disso, o projeto trouxe uma visão mais prática sobre modelagem de dados. Foi necessário pensar não apenas em tabelas isoladas, mas em como as informações se relacionam dentro do funcionamento real do Julius Hot Dogs, como pedidos consumindo produtos, produtos utilizando insumos e usuários interagindo com diferentes funcionalidades.

Também foi possível compreender que o desenvolvimento de sistemas para negócios reais envolve fatores além da tecnologia. Questões como fluxo de atendimento, horários de pico, controle de estoque e necessidades do empreendedor influenciam diretamente nas decisões técnicas. Esse contato com a realidade do negócio ajudou a desenvolver uma visão mais completa sobre análise, planejamento e desenvolvimento de software.

Por fim, o projeto reforçou a importância de testes, ajustes e evolução contínua durante o desenvolvimento. Erros encontrados durante a configuração do ambiente, conexão com banco de dados e

criação das rotas contribuíram para um aprendizado prático sobre resolução de problemas e adaptação, preparando a estrutura do backend para as próximas etapas do sistema.

Referências Bibliográficas

As referências abaixo foram selecionadas considerando os principais conceitos utilizados no desenvolvimento do backend do projeto **Julius Hot Dogs**, incluindo desenvolvimento de APIs REST, arquitetura de sistemas, banco de dados, autenticação, modelagem e boas práticas de programação.

NODE.JS FOUNDATION. Node.js Documentation.

Disponível em: <https://nodejs.org/docs/latest/api/>

Acesso em: 10 jun. 2026.

Referência utilizada para fundamentar o desenvolvimento do servidor backend, gerenciamento de módulos, execução assíncrona e comunicação entre a aplicação e os serviços externos.

EXPRESS.JS. Express - Node.js Web Application Framework.

Disponível em: <https://expressjs.com/>

Acesso em: 10 jun. 2026.

Utilizada como referência para implementação das rotas da API REST, middlewares, tratamento de requisições HTTP e organização da arquitetura backend.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation.

Disponível em: <https://www.postgresql.org/docs/>

Acesso em: 05 jun. 2026.

Referência utilizada para criação e gerenciamento do banco de dados relacional utilizado pelo sistema, incluindo tabelas, relacionamentos, consultas e integridade dos dados.

SUPABASE INC. Supabase Documentation.

Disponível em: <https://supabase.com/docs>

Acesso em: 09 jun. 2026.

Referência utilizada para integração com banco de dados PostgreSQL, autenticação, armazenamento de dados e serviços disponibilizados pela plataforma Supabase.

FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. University of California, Irvine, 2000.

Disponível em:

<https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Referência utilizada para compreensão do modelo REST, arquitetura aplicada na comunicação entre frontend, backend e serviços da aplicação.

FOWLER, Martin. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

Livro utilizado como referência para organização da arquitetura do software, separação de responsabilidades, camadas da aplicação e boas práticas no desenvolvimento backend.

SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2019.

Referência utilizada para conceitos relacionados ao ciclo de vida do software, levantamento de requisitos, modelagem, desenvolvimento e testes da aplicação.

DATE, C. J. *Introdução a Sistemas de Bancos de Dados*. 8. ed. Rio de Janeiro: Elsevier, 2004.

Utilizada como base teórica para modelagem relacional, normalização, relacionamentos entre tabelas e organização dos dados persistidos no sistema.

OWASP FOUNDATION. *OWASP Top 10 — Web Application Security Risks*.

Disponível em: <https://owasp.org/www-project-top-ten/>

Acesso em: 10 jun. 2026.

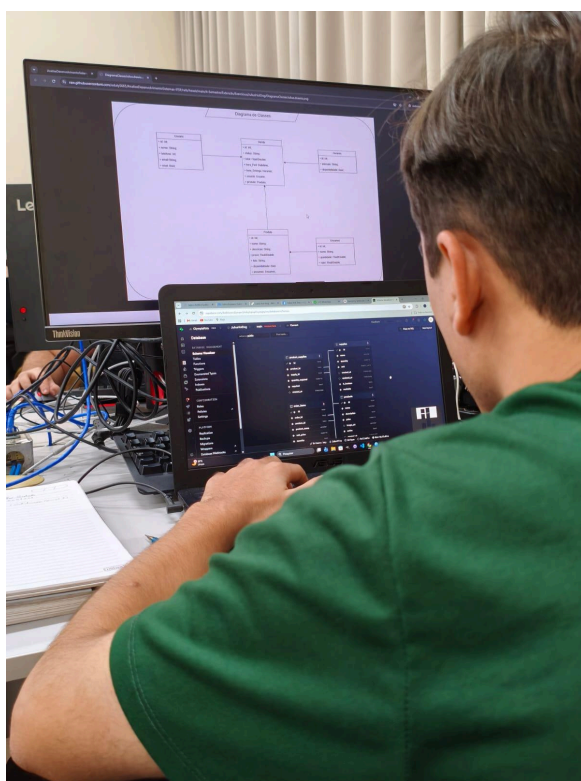
Referência utilizada para boas práticas de segurança no desenvolvimento web, proteção de dados, autenticação e prevenção contra vulnerabilidades.

PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: Uma Abordagem Profissional*. 9. ed. Porto Alegre: AMGH, 2021.

Referência utilizada para fundamentar processos de desenvolvimento, documentação, testes e manutenção do sistema.

Anexos:

I) Registro fotográfico



II) Repositório do sistema

Quando for o caso – endereço de repositório de códigos; local onde o sistema está disponível (como domínio ou site hospedado).

<https://julius-hot-dog.onrender.com/login>

Link do Projeto - Site Hospedado

<https://github.com/guilhermebatista7/JuliusHotDog>

Link do Projeto - Código Fonte Website

<https://github.com/eduty5665/AnaliseDesenvolvimentoSistemas-IFSP/tree/main/4-Semestre/Extens%C3%A3o/Exerc%C3%ADcios/JuliusHotDog>

Link do Projeto - Documentação e Diagramação